Programming with Lab Exercises and Application Project: Report

by Peter Ivanov [BSc (Hons) Software Engineering]

13.11.2025

Contents

Task A

Testing: Pytest

Unit Tests

```python
### ex 1
def test_calculator():
    assert calculator("qwe", 0, "+") ==  "Error: invalid input."
    assert calculator([], None, "") ==  "Error: invalid input."
    assert calculator(0, 0, "$") ==  "Error: invalid operator."
    assert calculator(0, 0, "+") ==  0
    assert calculator(-100000, 0.000001, "+") == -99999.999999
    assert calculator(10, 5, "-") == 5
    assert calculator(5, -5, "-") == 10
    assert calculator(5, -5, "*") == -25
    assert calculator(-5, -5, "*") == 25
    assert calculator(5, 0, "*") == 0
    assert calculator(5, 0, "/") == "Error: division by 0 is not allowed."
    assert calculator(5, 5, "/") == 1
    assert calculator(5, 4, "%") == 1
    assert calculator(2, 4, ">") == False
    assert calculator(2, 3, ">=") == False
    assert calculator(2, 4, "<") == True
    assert calculator(2, 2, "<") == False
    assert calculator(2, 2, "<=") == True

### ex 2
def test_max_of_three():
    assert max_of_three("qwe", 20, 30) == "Error: invalid input."
    assert max_of_three(10, 10, 30) == 30
    assert max_of_three(10, 10, 10) == 10
    assert max_of_three(-10, -20, -20) == -10

### ex 3
def test_winning_numbers():
    assert winning_numbers([5, 14, 17], [5, 14, 6]) == "Second."
    assert winning_numbers([1, 2, 3], [1, 2, 3]) == "First."
    assert winning_numbers([4, 7, 9], [0, 7, 2]) == "Third."
    assert winning_numbers([10, 20, 30], [1, 2, 3]) == "No."
    assert winning_numbers([10, 20, 30], [10, 10, 10]) == "Error: invalid amount of inputs or used duplicates."
    assert winning_numbers([[], 20, 30], ["qwe", 2, 3]) == "Error: only integers must be used."

### ex 4
def test_sum_of_evens():
    assert sum_of_evens("qwe", 20) == "Error: inputs must be integers."
    assert sum_of_evens(12, -20) == "Error: inputs must be positive."
    assert sum_of_evens(12, 21) == 80
    assert sum_of_evens(21, 12) == 80

### ex 5
def test_calculate_average():
    assert calculate_average([]) == "Error: empty list."
    assert calculate_average(["qwe", 2, 3]) == "Error: only numbers must be entered."
    assert calculate_average([1]) == 1
    assert calculate_average([1, 4, 5, 6]) == 4
    assert calculate_average([1, 1.5, 2]) == 2

### ex 6
def test_calculate_weekly_pay():
    assert calculate_weekly_pay("qwe") == "Error: input should be a positive integer."
    assert calculate_weekly_pay(0) == "Error: input should be a positive integer."
    assert calculate_weekly_pay(-1) == "Error: input should be a positive integer."
    assert calculate_weekly_pay(30) == 360
    assert calculate_weekly_pay(36) == 438
```

```python
### ex 7
def test_is_prime():
    assert is_prime(0) == "Error: input should be a positive integer."
    assert is_prime(-1) == "Error: input should be a positive integer."
    assert is_prime(1) == False
    assert is_prime(21) == False
    assert is_prime(5) == True

### ex 8
def test_are_anagrams():
    assert are_anagrams("", "EARTH") == "Error: input cannot be empty."
    assert are_anagrams(1, "EARTH") == "Error: input should only contain letters."
    assert are_anagrams("HEART", "earth") == True
    assert are_anagrams("help", "self") == False

### ex 9
def test_count_vowels():
    assert count_vowels("") == "Error: input cannot be empty."
    assert count_vowels(1) == 0
    assert count_vowels("Hello World") == 3
    assert count_vowels("Are you okay?") == 6

### ex 10
def test_sort_list():
    assert sort_list(1) == "Error: input should be a list."
    assert sort_list([]) == "Error: list cannot be empty."
    assert sort_list(["qwe", 2, 3]) == "Error: a list of numbers must be used."
    assert sort_list([5, 6, 9, 0]) == [0, 5, 6, 9]
    assert sort_list([1.2, 2, 1]) == [1, 1.2, 2]
    assert sort_list([-1, 9, -10]) == [-10, -1, 9]

### ex 11
def test_sum_of_digits():
    assert sum_of_digits("qwe") == "Error: input should be a positive integer."
    assert sum_of_digits(-1) == "Error: input should be a positive integer."
    assert sum_of_digits(1.2) == "Error: input should be a positive integer."
    assert sum_of_digits(0) == 0
    assert sum_of_digits(123) == 6

### ex 12
def test_is_palindrome():
    assert is_palindrome(123) == False
    assert is_palindrome("Radar") == True
    assert is_palindrome("python") == False

### ex 13
def test_password_strength():
    assert password_strength("") == "Error: password cannot be empty."
    assert password_strength("abc") == "Weak password."
    assert password_strength(123) == "Weak password."
    assert password_strength("Abc@12") == "Medium password."
    assert password_strength("MyPass$2025") == "Strong password."
```

```python
### ex 14
def test_letter_grade():
    data_input = [
        {"subject": "Math", "score": 90, "credits": 40},
        {"subject": "History", "score": 75, "credits": 20},
        {"subject": "English", "score": 60, "credits": 20},
        {"subject": "Science", "score": 85, "credits": 40},
        {"subject": "Art", "score": 50, "credits": 40}
    ]
    assert letter_grade({}) == "Error: input cannot be empty."
    assert letter_grade(data_input) == (73.125, "B")

    data_input[0]["score"] = -1
    assert letter_grade(data_input) == "Error: score needs to be between 0 and 100."

    data_input = [
        {"subject": "Math", "score": 90, "credits": 0},
        {"subject": "History", "score": 75, "credits": 0}
    ]
    assert letter_grade(data_input) == "Error: total credits cannot be zero."

### ex 15
def test_maximum_gap():
    assert maximum_gap("qwe", [1, 2]) == "Error: enter two lists of integers."
    assert maximum_gap([], [1, 2]) == "Error: enter two lists of integers."
    assert maximum_gap([1, 2.2], [1, 2]) == "Error: enter two lists of integers."
    assert maximum_gap([1, 5, 600], [100, 7, 3, 29, 39]) == 597
    assert maximum_gap([1, 5, 600], [100, 7, 3, 602, 39]) == 601

### ex 16
def test_cipher_text():
    cipher = "Hdfk#huuru#|rx#pdnh#lq#surjudpplqj#lv#dq#rssruwxqlw|#wr#ehfrph#d#ehwwhu#ghyhorshu$"
    key = 3
    assert cipher_text(cipher, key) == "Each error you make in programming is an opportunity to become a better developer!"
    key += 256
    assert cipher_text(cipher, key) == "Each error you make in programming is an opportunity to become a better developer!"
    assert cipher_text(cipher, 0) == "Error: key should be a positive integer."
    assert cipher_text(cipher, "qwe") == "Error: key should be a positive integer."

### ex 17
def test_net_annual_income():
    assert net_annual_income(-1) == "Error: input should be a positive number."
    assert net_annual_income(0) == "Error: input should be a positive number."
    assert net_annual_income(60000) == 48568

### ex 18
def test_my_split():
    assert my_split("apple,banana,orange", ",") == ["apple", "banana", "orange"]
    assert my_split("apple banana orange", " ") == ["apple", "banana", "orange"]

### ex 19
def test_longest_repetition():
    assert longest_repetition("") == ("", 0)
    assert longest_repetition(123) == ("1", 1)
    assert longest_repetition("abc") == ("a", 1)
    assert longest_repetition("aaabbcaaa") == ("a", 3)
    assert longest_repetition("hellooooo") == ("o", 5)

### ex 20
def test_closest_pair_under_budget():
    items = [("tv", 300), ("mobile phone", 800), ("laptop", 600), ("headphones", 200)]
    budget = 1000
    assert closest_pair_under_budget(items, budget) == ["mobile phone", "headphones"]
    assert closest_pair_under_budget(items, 400) == None
    assert closest_pair_under_budget([], budget) == "Error: input cannot be empty."
    assert closest_pair_under_budget(items, -10) == "Error: budget must be a positive number."
    assert closest_pair_under_budget(("tv", 300), budget) == "Error: items must be presented in a list."
    assert closest_pair_under_budget([("tv", -10), ("mobile phone", 800)], budget) == "Error: price should be a positive number."
    assert closest_pair_under_budget([["tv", -10], ("mobile phone", 800)], budget) == "Error: invalid list of items."
```

Test Results

```
============================= test session starts =============================
platform win32 -- Python 3.13.5, pytest-8.3.4, pluggy-1.5.0
rootdir: C:\Users\whyno\OneDrive - Bournemouth University\Programming\Assessment
plugins: anyio-4.7.0
collected 20 items

test_lab_ex.py ....................                                      [100%]

============================== 20 passed in 0.24s ==============================
```

Task B

Design: Pseudocode & Structured English

Attached pseudocode.txt: link

```
dict item1 = {name: Water, price: 1.00, stock: 10}
dict item2 = {name: Soda, price: 1.20, stock: 8}
dict item3 = {name: Energy Drink, price: 2.00, stock: 5}
dict item4 = {name: Cookie, price: 0.50, stock: 2}
dict item5 = {name: Crisps, price: 1.50, stock: 0}

list menu = item1, item2, item3, item4, item5
int order_num = random integer between 100 and 1000
set allowed_coins = {0.20, 0.50, 1.00, 2.00}
float balance = 0
list items_bought = []
float total_spent = 0
float true_total = 0
list change_given = []
float total_money_added = 0
str filename = ""

main()

FUNC main():
    WHILE True:
        insert_coins()
        purchasing()
        reset_vars()
        OUTPUT "Next customer..."

FUNC insert_coins():
    WHILE True:
        OUTPUT "Current balance: £{balance}"
        INPUT "Insert a coin ({allowed_coins}) or type 'done' to finish: "
        IF INPUT == "done":
            BREAK
        ELSE IF NOT check_input_coin(INPUT):
            OUTPUT "Error: invalid input."

FUNC check_input_coin(input):
    TRY:
        IF input in allowed_coins:
            balance += input
            total_money_added += input
            RETURN True
    EXCEPT:
        RETURN False

FUNC purchasing():
    WHILE True:
        print_menu()
        INPUT "Select an item by name, or type 'add' to top up your balance, or type
'exit' to quit: "
        IF INPUT == "exit":
            finish()
            BREAK
        ELSE IF INPUT == "add":
            insert_coins()
```

```
        ELSE:
            IF check_input_purchase(INPUT):
                dict current_item = chosen item from menu
                IF current_item[price] > balance:
                    OUTPUT "Error: insufficient balance."#
                ELSE IF current_item[stock] == 0:
                    OUTPUT "Sorry, we are out of stock. Choose another item."
                ELSE:
                    list temp_items_bought = items_bought + [current_item]
                    temp_true_total = true_total + current_item[price]
                    float discount = check_discount(temp_true_total, temp_items_bought)
                    float spent_now = current_item[price] * discount
                    current_item[discount] = ROUND (100 * (1 - discount))
                    balance -= spent_now
                    true_total += current_item[price]
                    total_spent += spent_now
                    APPEND current_item TO items_bought
                    current_item["stock"] -= 1
                    OUTPUT "Dispensing item..."
                    OUTPUT "Purchased {current_item[name]} for £{spent_now} (discount:
{ROUND (100 * (1 - discount))}%)"
            ELSE:
                OUTPUT "Error: invalid input."

FUNC check_discount(temp_true_total, temp_items_bought):
    float total = current_total IF current_total IS NOT None ELSE total_spent
    list items = current_items IF current_items IS NOT None ELSE items_bought
    IF total > 7:
        RETURN 0.85
    ELSE IF total > 5:
        RETURN 0.9
    ELSE IF LENGTH (items) >= 3:
        RETURN 0.95
    ELSE:
        RETURN 1

FUNC check_input_purchase(input):
    IF input in menu:
        RETURN True
    RETURN False

FUNC print_menu():
    OUTPUT "Vending Machine Menu"
    OUTPUT all items in menu

FUNC finish():
    REVERSE allowed_coins
    FOR coin IN allowed_coins:
        WHILE balance >= coin:
            APPEND coin TO change_given
            balance -= coin
    bool bonus = False
    IF order_num % 2 == 1 AND total_spent > 2:
        bonus = True
        APPEND 1.0 to change_given
    IF change_given:
        OUTPUT "Dispensing change:"
        FOR coin IN change_given:
            OUTPUT "£{coin}"
        IF bonus:
            OUTPUT "[Bonus change of £1.00 due to odd-numbered order (#{order_num}) and
spending over £2.00]"
```

```
    ELSE:
        OUTPUT "No change to dispense."
    create_receipt()
    OUTPUT "Thank you for the purchase! Have a great day!"

FUNC create_receipt():
    str filename = "receipt_{order_num}.csv"
    WRITE INTO FILE:
        WRITEROW "ORDER NUMBER: #{order_num}"
        WRITEROW datetime.now()
        WRITEROW "Count", "Item", "Price (£)", "Discount", "Final Price (£)"
        int count = 1
        FOR item IN items_bought:
            WRITEROW count, item[name], item[price], item[discount], item[price] * (1 -
item[discount]/100)
            count += 1
        WRITEROW "TOTAL SPENT", total_spent
        WRITEROW "MONEY ADDED", total_money_added
        WRITEROW "CHANGE GIVEN", SUM (change_given)
    OUTPUT "Receipt created for order #{order_num}."

FUNC reset_vars():
    order_num += 1
    balance = 0
    items_bought = []
    total_spent = 0
    true_total = 0
    change_given = []
    total_money_added = 0
```

Progress: Description & Screenshots

1) Allow the user to insert coins and keep track of balance (*Figure 1*)

```python
allowed_coins = [0.20, 0.50, 1.00, 2.00]
balance = 0.00

def insert_coins():
    global balance
    while True:
        print(f"Current balance: £{balance:.2f}")
        coin = input("Insert coin or 'done': ")
        if coin == "done":
            break
        elif float(coin) in allowed_coins:
            balance += float(coin)
        else:
            print("Invalid coin.")

insert_coins()
```

Figure 1

1) Display available items with prices and stock (*Figure 2*)

```python
item1 = {"name": "Water", "price": 1.00, "stock": 10}
item2 = {"name": "Soda", "price": 1.20, "stock": 8}
item3 = {"name": "Energy Drink", "price": 2.00, "stock": 5}
item4 = {"name": "Cookie", "price": 0.50, "stock": 2}
item5 = {"name": "Crisps", "price": 1.50, "stock": 0}

menu = [item1, item2, item3, item4, item5]

def print_menu():
    print("-------- Vending Machine Menu --------"); print()

    i = 1
    for item in menu:
        print(f"{i}. {item["name"]} - £{item["price"]:.2f}   ({item["stock"]} in stock)")
        i += 1

    print(); print("---------------------------------------")

print_menu()
```

Figure 2

2) Allow the user to select an item and deduct from balance (*Figure 3*)

```python
def purchase():
    global balance
    print_menu()
    choice = input("Select item name: ")
    for item in menu:
        if item["name"].lower() == choice.lower():
            if balance >= item["price"]:
                balance -= item["price"]
                item["stock"] -= 1
                print(f"Purchased {item['name']}")
            else:
                print("Not enough balance.")

```

Figure 3

3) Introduce discounts (*Figure 4*)

```python
def check_discount(total):
    if total > 7:
        return 0.85
    elif total > 5:
        return 0.9
    elif len(items_bought) >= 3:
        return 0.95
    else:
        return 1.0
```

Figure 4

4) Introduce CSV receipts (*Figure 5*)

```python
def create_receipt():
    filename = f"receipt_{order_num}.csv"
    with open(filename, "w", newline="") as file:
        writer = csv.writer(file)
        writer.writerow(["Item", "Price", "Discount", "Final Price"])
        for item in items_bought:
            writer.writerow([item["name"], item["price"], item["discount"], item["price"] * (1 - item["discount"]/100)])
```

Figure 5

5) Add change dispensing (*Figure 6*)

```python
def finish():
    global change_given, balance, allowed_coins

    allowed_coins.sort(reverse = True)

    for coin in allowed_coins:
        while balance >= coin - 0.001:
            change_given.append(coin)
            balance -= coin

    bonus = False
    if order_num % 2 == 1 and total_spent > 2:
        bonus = True
        change_given.append(1.0)

    if change_given:
        print(); print("Dispensing change:", flush = True)
        for coin in change_given:
            time.sleep(1)
            print(f"£{coin:.2f}", flush = True)
        if bonus:
            time.sleep(1)
            print(); print(f"[Bonus change of £1.00 due to odd-numbered order (#{order_num}) and spending over £2.00]")
    else:
        print(); print("No change to dispense.")

    create_receipt()

    print(); print("Thank you for the purchase! Have a great day!")
```

Figure 6

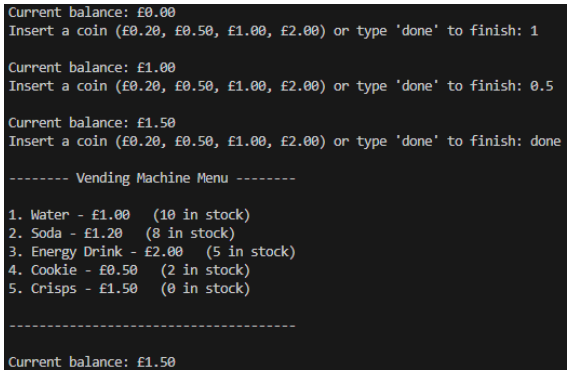
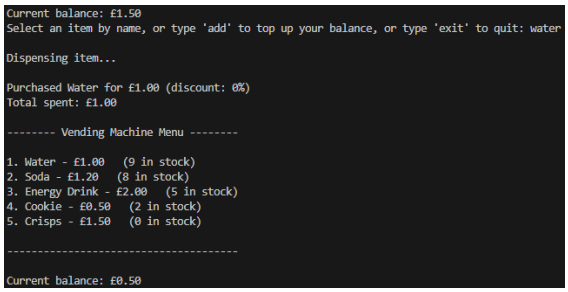
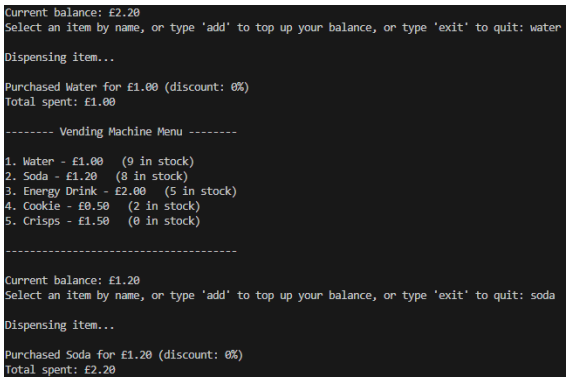
6) Add full program loop (*Figure 7*)

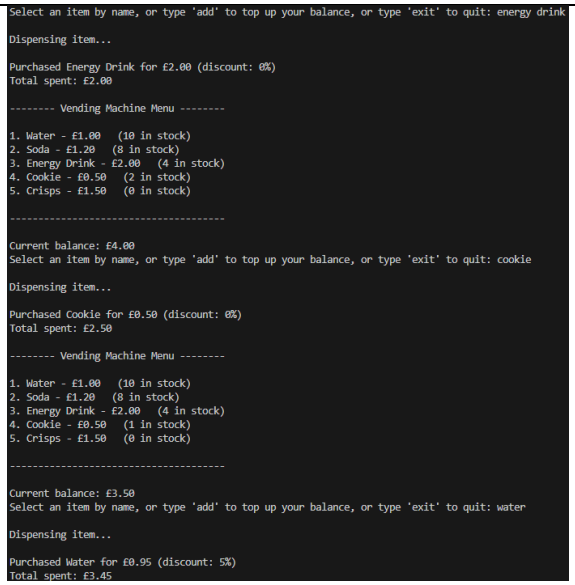
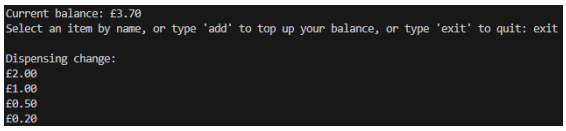
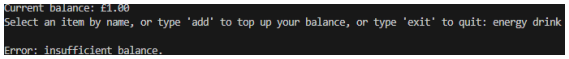
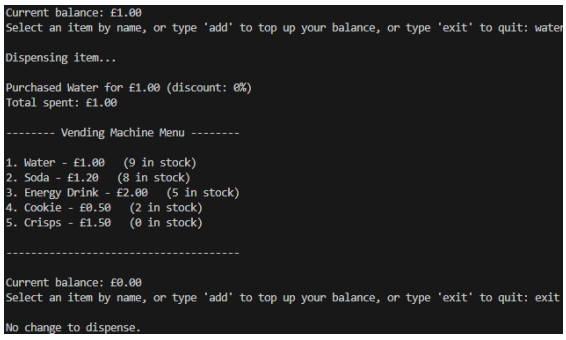
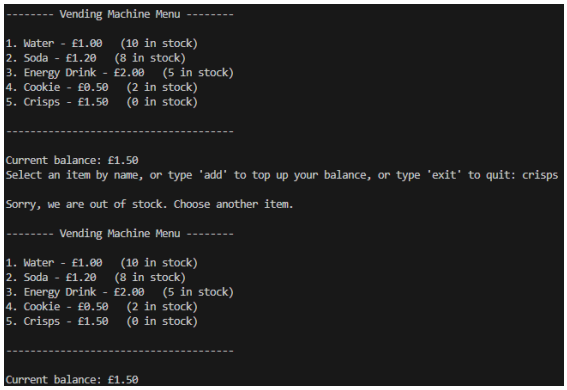
```python
def main():
    while True:
        insert_coins()
        purchasing()
        reset_vars()
        print(); print("Next customer...")


if __name__ == "__main__":
    main()
```

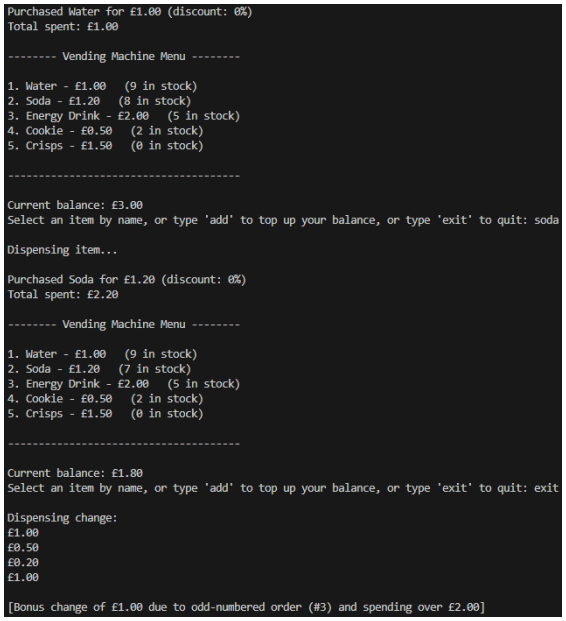
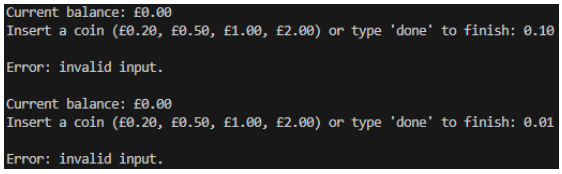
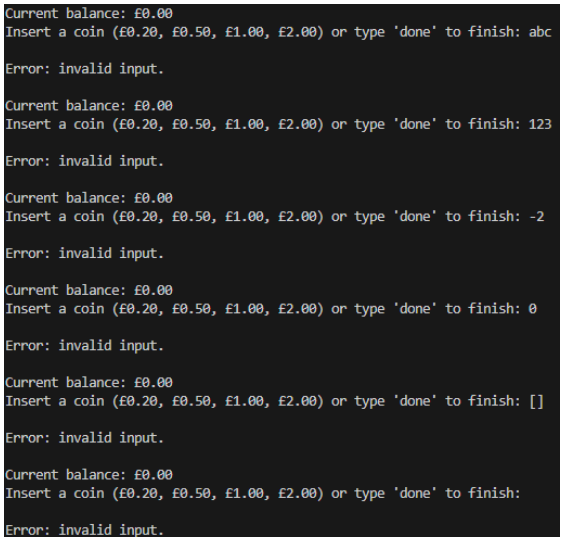
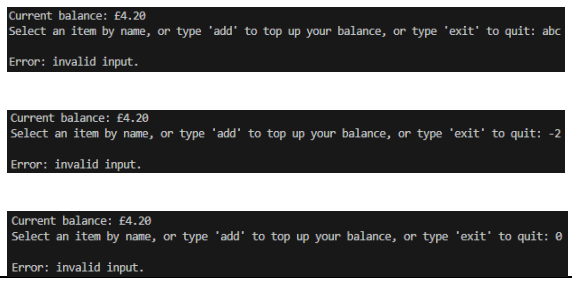
Figure 7

Testing: Normal, Edge-case, Invalid Input

Table

Description	Input / Action	Expected Result	Screenshot
Insert valid coins	Insert £1.00, £0.50, then 'done'	Balance = £1.50	
Buy available item	Buy Water	Stock of Water decreases by 1, balance reduces by £1.00	
Multiple purchases	Buy Water and Soda	Correct total spent of £2.20	

Apply discount	Buy 3 items	Discount of 5% applied on last item	
Change dispensed	Balance = £3.70, then ‘exit’	Dispensed change: £2.00, £1.00, £0.50, and £0.20	
Insufficient balance	Balance = £1.00, buy Energy Drink for £2.00	Error: insufficient balance	
Exact balance	Balance = £1.00, buy Water, then ‘exit’	Successful purchase, no change dispensed	
Out of stock	Buy Crisps (stock 0)	Error: out of stock, balance doesn’t change	

Trigger bonus	Order number is odd, spend > £2.00	£1.00 bonus added to change	
Invalid coin	Insert £0.10, £0.01	Error: invalid input	
Invalid input when inserting coins	Type ‘abc’, 123, -2, 0, [], empty input	Error: invalid input	
Invalid input when selecting item	Type ‘abc’, 123, -2, 0, [], empty input	Error: invalid input	

			Current balance: £4.20 Select an item by name, or type 'add' to top up your balance, or type 'exit' to quit: [] Error: invalid input. Current balance: £4.20 Select an item by name, or type 'add' to top up your balance, or type 'exit' to quit: Error: invalid input.
Creating receipt	Order number is 3 (odd); add £4.00; buy Water, Energy Drink, and Cookie; type 'exit'	New CSV file saved with correct name and data	 *Link to CSV file: link*
Next customer	Finish order	Order number increases by 1; stock stays the same; variables get reset (e.g. balance)	Current balance: £0.00 Insert a coin (£0.20, £0.50, £1.00, £2.00) or type 'done' to finish: 2 Current balance: £2.00 Insert a coin (£0.20, £0.50, £1.00, £2.00) or type 'done' to finish: done -------- Vending Machine Menu -------- 1. Water - £1.00 (10 in stock) 2. Soda - £1.20 (8 in stock) 3. Energy Drink - £2.00 (5 in stock) 4. Cookie - £0.50 (2 in stock) 5. Crisps - £1.50 (0 in stock) ---------- Current balance: £2.00 Select an item by name, or type 'add' to top up your balance, or type 'exit' to quit: energy drink Dispensing item... Purchased Energy Drink for £2.00 (discount: 0%) Total spent: £2.00 -------- Vending Machine Menu -------- 1. Water - £1.00 (10 in stock) 2. Soda - £1.20 (8 in stock) 3. Energy Drink - £2.00 (4 in stock) 4. Cookie - £0.50 (2 in stock) 5. Crisps - £1.50 (0 in stock) ---------- Current balance: £0.00 Select an item by name, or type 'add' to top up your balance, or type 'exit' to quit: exit No change to dispense. Receipt created for order #834. Thank you for the purchase! Have a great day! Next customer... Current balance: £0.00 Insert a coin (£0.20, £0.50, £1.00, £2.00) or type 'done' to finish: 1 Current balance: £1.00 Insert a coin (£0.20, £0.50, £1.00, £2.00) or type 'done' to finish: done -------- Vending Machine Menu -------- 1. Water - £1.00 (10 in stock) 2. Soda - £1.20 (8 in stock) 3. Energy Drink - £2.00 (4 in stock) 4. Cookie - £0.50 (2 in stock) 5. Crisps - £1.50 (0 in stock) ---------- Current balance: £1.00 Select an item by name, or type 'add' to top up your balance, or type 'exit' to quit: water Dispensing item... Purchased Water for £1.00 (discount: 0%) Total spent: £1.00 -------- Vending Machine Menu -------- 1. Water - £1.00 (9 in stock) 2. Soda - £1.20 (8 in stock) 3. Energy Drink - £2.00 (4 in stock) 4. Cookie - £0.50 (2 in stock) 5. Crisps - £1.50 (0 in stock) ---------- Current balance: £0.00 Select an item by name, or type 'add' to top up your balance, or type 'exit' to quit: exit No change to dispense. Receipt created for order #835. Thank you for the purchase! Have a great day! Next customer...

Conclusion

The program successfully handled both normal and edge cases. Invalid input did not cause crashes, and everything worked as expected.

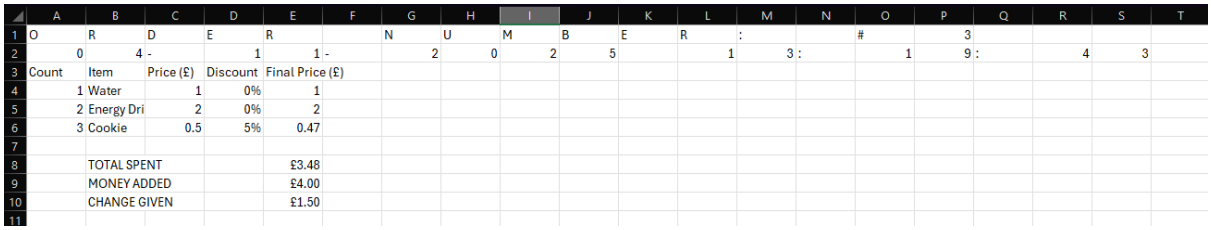
Reflection

This coursework was more challenging than I expected and taught me important lessons about writing reliable code that users can work with.

I have gotten much better with the testing process. Using normal, edge-case and invalid inputs helped me fix many bugs I would have overlooked. It was also my first time using Pytest, a tool I found useful, which I am going to continue using in my future projects.

Completing all lab exercises was a challenge. It took a lot of patience, as each task is unique and has special requirements. However, I view it as a good thing, as I had a great chance to practice some tricky parts of Python, and, more importantly, consolidate the fundamentals. I learnt how to make the code robust by validating input, and how to solve complex real-life problems.

The vending machine project was a challenge too. The greatest one was also validating user input. When I typed "abc" instead of a coin value, my code threw a *ValueError* and terminated, and when typed "water" instead of "Water", the application failed too. I resolved this by creating dedicated validation functions. And these are just two examples, and with this kind of program there are many. At first, it was complicated to think of every possible error that might occur during runtime, but it got much easier and more intuitive with the development. Another challenge was applying discounts. The final application includes several tweaks compared to the initial one, which did not work the way I wanted it to. For example, when buying three items, a discount of 5% would only apply to the next item. I moved the discount calculation to a separate function called *check_discount(),* that worked on a temporary total, making sure the discount was applied to the current purchase. The final design correctly applies a 15% discount when spending £7 or more, 10% for spending over £5, and 5% when buying three or more items. This change was necessary for the system to be fair and predictable. I also learnt how to implement the receipt feature, using Python's CSV library. One issue I couldn't overcome, is each character in the header was enclosed in a separate cell, when viewing in Excel, or separated by a comma, when viewing as text, as shown in *Figure 8* and *Figure 9* respectively (next page).



	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
1	O	R	D	E	R		N	U	M	B	E	R	:		#	3				
2	0	4	-	1	1	-	2	0	2	5	1	3	:	1	9	:	4	3		
3	Count	Item	Price (£)	Discount	Final Price (£)															
4	1	Water	1	0%	1															
5	2	Energy Dri	2	0%	2															
6	3	Cookie	0.5	5%	0.47															
7																				
8		TOTAL SPENT			£3.48															
9		MONEY ADDED			£4.00															
10		CHANGE GIVEN			£1.50															
11																				

Figure 8

```
O,R,D,E,R, ,N,U,M,B,E,R,:, ,#,3
0,4,-,1,1,-,2,0,2,5, ,1,3,:,1,9,:,4,3
Count,Item,Price (£),Discount,Final Price (£)
1,Water,1.00,0%,1.00
2,Energy Drink,2.00,0%,2.00
3,Cookie,0.50,5%,0.47

,TOTAL SPENT,,,£3.48
,MONEY ADDED,,,£4.00
,CHANGE GIVEN,,,£1.50
```

Figure 9

AI Use

OpenAI. (2025). *ChatGPT* (November 4 version) [Large language model]. https://chat.openai.com

Prompt: [attached code] how to do csv receipts in python

Response: *link*

Adaptation: I used the CSV library, I changed the structure, variable names, and formatting to fit my program's needs.